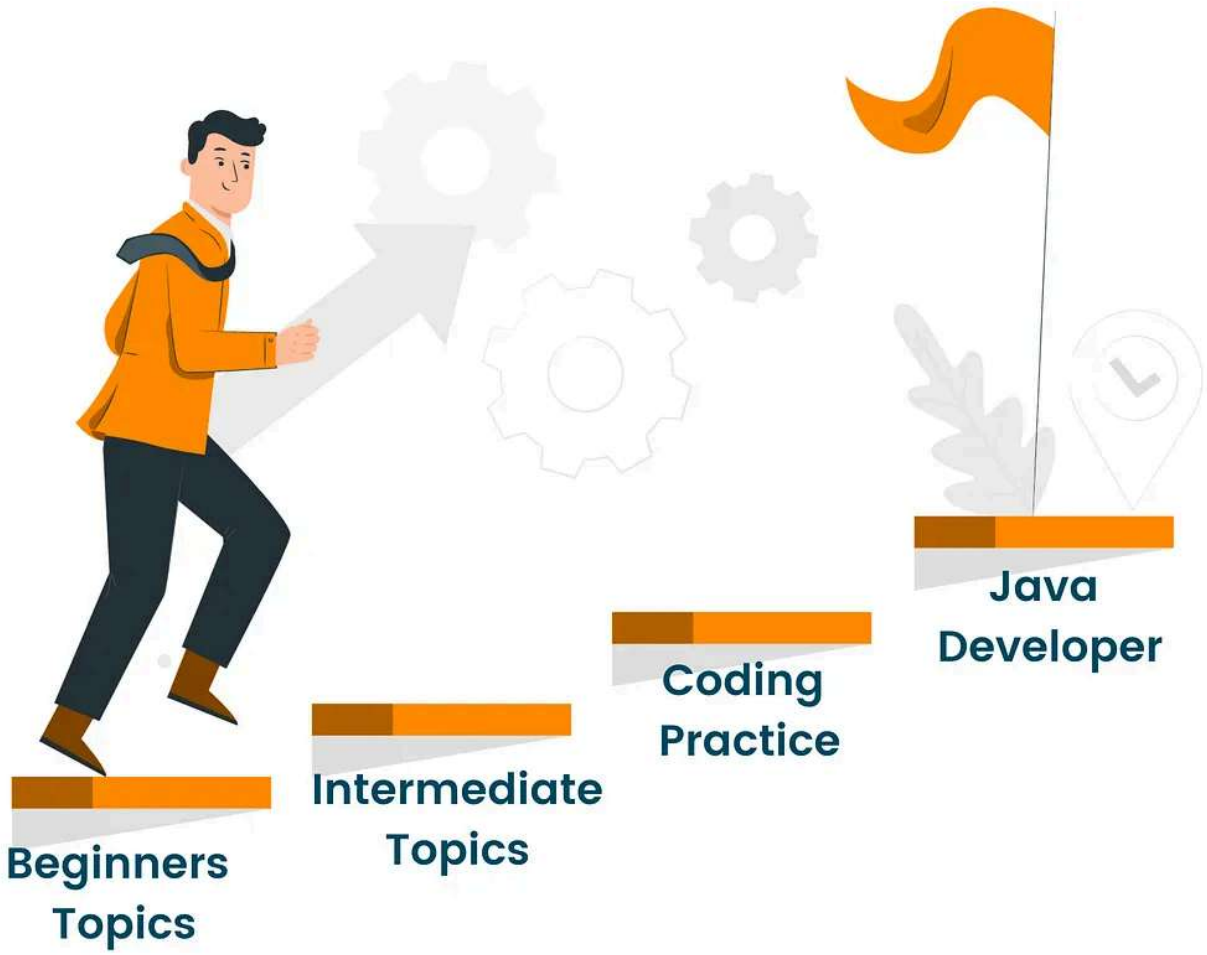
Open in app ↗ Sign up Sign in

Medium Search Write



Complete Core Java Developer Roadmap with all Topics.

1. Introduction to Java

Amarjeet Kumar Singh Follow 4 min read · Aug 25, 2024

- **Overview of Java**: History, features, and use cases.
- **Setting Up Java Environment**: Install JDK, setup IDE (Eclipse, IntelliJ IDEA, or VS Code).
- **Hello World Program**: Writing your first Java program.

2. Basic Syntax and Structure

- **Java Syntax**: Keywords, identifiers, literals.
- **Identifiers**: Naming conventions, rules for identifiers.
- **Data Types**: Primitive types (int, char, boolean, etc.) and reference types.
- **Variables**: Declaration, initialization, and scope.
- **Operators**: Arithmetic, relational, logical, bitwise, assignment, and miscellaneous operators.
- **Control Flow Statements**:
- **Conditional Statements**: if, else if, else, switch-case.
- **Loops**: for, while, do-while, enhanced for-loop.
- **Keywords**: Understanding special keywords like `return`, `break`, `continue`, `instanceof`.

3. Arrays

- **Introduction to Arrays**: Single-dimensional and multi-dimensional arrays.
- **Array Operations**: Traversing, sorting, and searching arrays.

4. Strings and String Manipulation

- **String Class**: Creating and manipulating strings.

- **StringBuffer**: Mutable strings.
- **StringBuilder**: Fast mutable strings.

5. Methods

- **Defining Methods**: Method declaration, parameters, return values.
- **Method Overloading**: Same method name with different parameters.
- **Method Parameters**: Pass-by-value, varargs.

6. Type Casting

- **Primitive Type Casting**: Implicit and explicit casting.
- **Reference Type Casting**: Upcasting and downcasting.

7. Wrapper Classes

- **Wrapper Classes**: Understanding `Integer`, `Double`, `Character`, `Boolean`, etc.
- **Autoboxing and Unboxing**: Automatic conversion between primitive types and wrapper classes.

8. Core Classes

- **Object Class**: Methods provided by `Object` class like `toString()`, `equals()`, `hashCode()`.
- **Math Class**: Utility methods for mathematical operations.

9. Object-Oriented Programming (OOP)

- **Classes and Objects**: Defining classes, creating objects, constructors.
- **Constructors**: Default constructor, parameterized constructor, constructor overloading.

- **Inheritance**: extends keyword, method overriding, `super` keyword.
- **Polymorphism**: Compile-time (method overloading), runtime (method overriding), upcasting, and downcasting.
- **Abstraction**: Abstract classes and interfaces.
- **Encapsulation**: Access modifiers (private, protected, public, default).
- **Data Hiding**: Private fields and methods, getter and setter methods.

10. Advanced OOP Concepts

- **Static Members**: Static variables, static methods, and static blocks.
- **Final Keyword**: Final classes, methods, and variables.
- **This Keyword**: Referring to the current instance.
- **Super Keyword**: Accessing parent class members.
- **Nested and Inner Classes**: Member inner class, static nested class, local inner class, and anonymous inner class.

11. Packages and Import

- **Packages**: Creating and using packages, importing classes, `java.lang` package.

12. Exception Handling

- **Types of Exceptions**: Checked, unchecked, and errors.
- **Try-Catch Block**: Writing exception handling blocks.
- **Finally Block**: Ensuring resource release.
- **Throw and Throws**: Declaring exceptions and throwing exceptions.
- **Custom Exceptions**: Creating your own exception classes.

13. Collections Framework

- **Introduction to Collections**: Benefits and hierarchy of collections.
- **List Interface**: ArrayList, LinkedList, Vector, Stack.
- **Set Interface**: HashSet, LinkedHashSet, TreeSet.
- **Map Interface**: HashMap, LinkedHashMap, TreeMap, Hashtable.
- **Queue Interface**: PriorityQueue, Deque.
- **Collections Class**: Utility methods for collections.
- **Iterators**: Using Iterator and ListIterator.

14. Java Generics

- **Introduction to Generics**: Benefits and basic syntax.
- **Generic Classes**: Defining classes with generic types.
- **Generic Methods**: Creating methods with generic types.
- **Bounded Type Parameters**: Using `<T extends Class>` in generics.
- **Wildcard in Generics**: `? extends` and `? super` keywords.

15. Java I/O Streams

- **Introduction to Streams**: Byte streams vs. character streams.
- **File Handling**: File, FileReader, FileWriter, BufferedReader, BufferedWriter.
- **Serialization and Deserialization**: Using `Serializable` interface.
- **Object Streams**: ObjectInputStream and ObjectOutputStream.
- **NIO (New I/O)**: Overview of java.nio package, buffers, channels.

16. Multithreading and Concurrency

- **Introduction to Threads**: Creating threads using `Thread` class and `Runnable` interface.
- **Thread Lifecycle**: States of a thread, thread methods.
- **Synchronization**: Synchronized methods and blocks, static synchronization.
- **Inter-Thread Communication**: `wait()`, `notify()`, `notifyAll()`.
- **Thread Pools**: Executor framework.
- **Concurrency Utilities**: Locks, semaphores, countdown latches, concurrent collections.

17. Java 8 Features

- **Lambda Expressions**: Syntax and usage.
- **Functional Interfaces**: Predicate, Function, Consumer, Supplier.
- **Stream API**: Creating streams, intermediate and terminal operations, parallel streams.
- **Optional Class**: Handling null values.
- **Default and Static Methods**: In interfaces.
- **Method References**: Reference to a static method, instance method, and constructor.

18. Best Practices and Design Patterns

- **Java Coding Best Practices**: Naming conventions, code organization, and common pitfalls.
- **Design Patterns**: Singleton, Factory, Strategy, Observer, etc.

- **Code Optimization**: Profiling, memory management, garbage collection tuning.

19. Build Tools and Project Management

- **Maven**: Dependency management, building projects.
- **Gradle**: Build automation.
- **JAR Files**: Creating and using JAR files.

20. Testing in Java

- **JUnit**: Writing unit tests, annotations, assertions.
- **Mockito**: Mocking objects, writing test doubles.
- **Test-Driven Development (TDD)**: Writing tests before code.

21. Working with Databases

- **JDBC**: Connecting to databases, executing queries, handling results.
- **ORM Frameworks**: Introduction to Hibernate, JPA (optional for core).

22. IDE Features and Debugging

- **Using IDEs Effectively**: Code generation, debugging, refactoring.
- **Debugging Techniques**: Breakpoints, watches, step execution.

23. Version Control

- **Git Basics**: Cloning, committing, branching, merging.
- **Working with Git Repositories**: Pushing to and pulling from remote repositories.

24. Projects and Practice

- **Mini Projects:** Implement a small project, such as a library management system, bank application, or to-do list.
- **Contribute to Open Source:** Contributing to Java projects on GitHub.

Learning Resources

- **Books:** “Effective Java” by Joshua Bloch, “Java: The Complete Reference” by Herbert Schildt.
- **Online Courses:** Platforms like Udemy, Coursera, or YouTube.
- **Documentation:** Official Java Documentation (https://docs.oracle.com/en/java/).

This roadmap provides a comprehensive guide for mastering Core Java, from basic concepts to advanced features. As you progress, make sure to build projects, practice coding, and explore additional resources to deepen your understanding.

Get Amarjeet Kumar Singh’s stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

Remember me for faster sign in

Thanks, before you go:

- 👏 Please clap for the story and follow the author 👉
- Please share your questions or insights in the comments section below. Let’s help each other and become better Java developers.

- Let's connect on https://www.linkedin.com/in/amarjeet-kumar-singh-956b46171/

Core Java | Java | Java Developer | Collection | Maps

Written by Amarjeet Kumar Singh

21 followers · 2 following

Follow

Experienced Java Full Stack Developer, Java Trainer, and Freelancer, Explore my work and Join our Publications:👉 https://medium.com/@amarjeetcs79

No responses yet

Write a response

What are your thoughts?

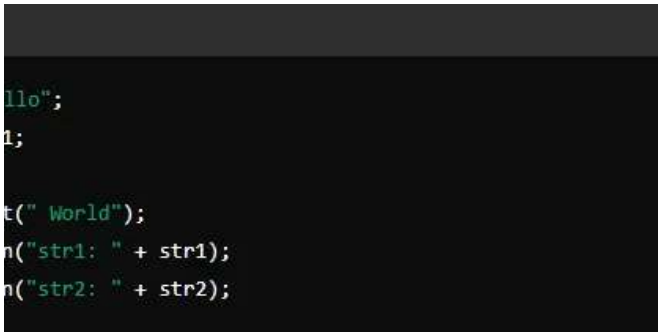
More from Amarjeet Kumar Singh

Amarjeet Kumar Singh · Aug 25, 2024

What is Angular Interpolation: Types, Syntax, Uses, and Practica...

In Angular, interpolation is a way to bind data from the component class to the template...

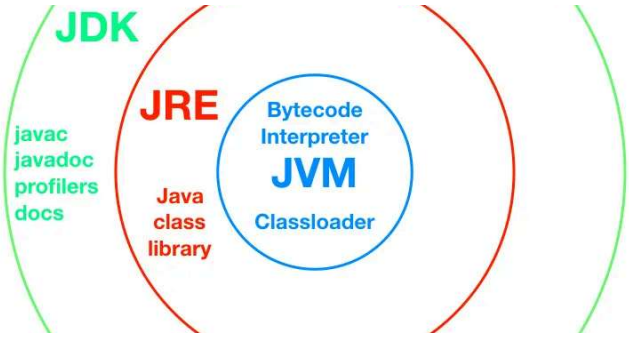
10



Amarjeet Kumar Singh · Aug 28, 2024

Top 10 String scenario-based interview questions for 1–3 years...

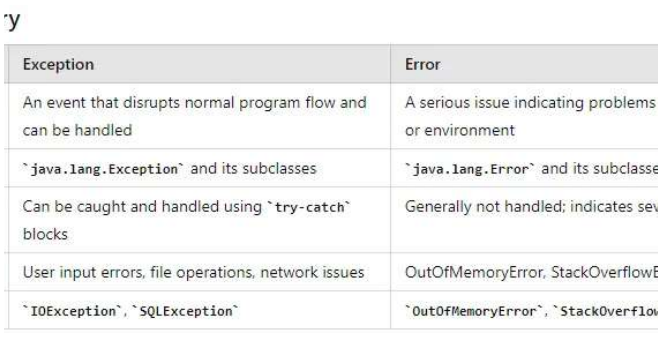
Scenario 1: String Immutability



Amarjeet Kumar Singh · Aug 26, 2024

Difference between JDK, JRE, and JVM in Java?

JDK (Java Development Kit)



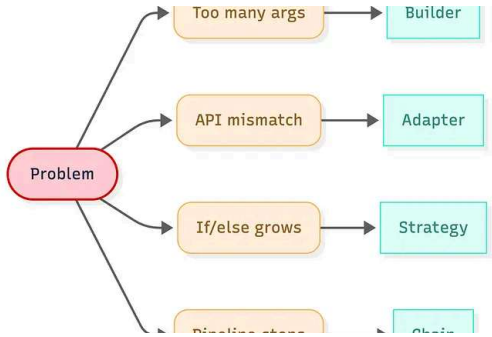
Amarjeet Kumar Singh · Aug 27, 2024

Difference between Exception & Error in Java?

In Java, exceptions and errors are two different types of problems that can occur...

See all from Amarjeet Kumar Singh

Recommended from Medium

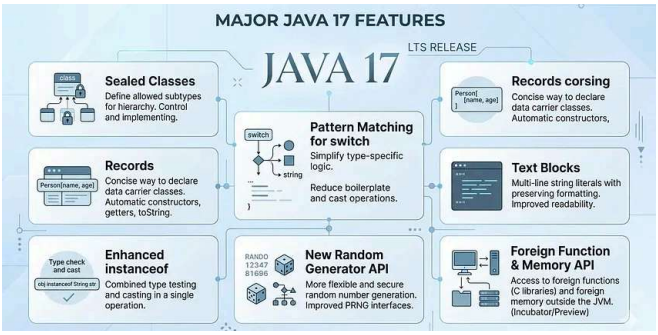


In Women in Technolo... by Alina Kovtun... · Jan 29

Stop Memorizing Design Patterns: Use This Decision Tree Instead

Choose design patterns based on pain points: apply the right pattern with minimal over-...

8.7K 90 57



Vinotech · May 21

Java 17 Features and Interview Questions and Answers

Java 17 is a Long-Term Support (LTS) release with many modern language, JVM, and API...

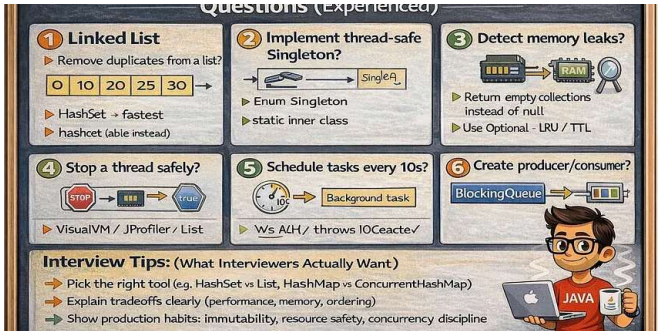
8

Ajay Rathod · Jan 11

Here Is My 400 Java Developer Interview Questions: Go to List to...

The Ultimate Java Developer Interview Guide: 400+ Questions to Land Your Dream Job

246 7 2



Gaini Laxman · Jan 13

Top 20 Java Scenario-Based Interview Questions for...

Scenario-based Java interview questions are designed to test one thing: Can you apply...

23 2

In Skill Stuff by CodeByUmar · Jun 8

7 Coding Patterns I Stole From Senior Engineers

Most developers do not become better because they learn more syntax. They...

3.1K 45 48

Pratiksha · Mar 4

Citi Bank Java Interview Question Round 1

Preparation Kit

36 1

See more recommendations